

Section 08: Sophos Firewall Integration (Perimeter Layer)

This section integrates your actual internship device into the lab, replacing the "optional pfSense" placeholder with a real NGFW as the first sensor in the pipeline. It also supersedes and corrects the Sophos-specific content in the uploaded `sophos_soc_threatintel_refined` draft — every correction below is explained, not just applied silently (see `AUDIT-LOG.md` for the full findings this section fixes).

Confidentiality, stated once here and meant for the whole project: your offer letter's confidentiality clause covers client credentials, firewall configs, network topology, and IP schemes observed during training. Nothing in this file — no IP, no screenshot, no rule — should ever come from an actual client site. Build and screenshot everything against your own lab instance (physical unit at home, or the free Home Edition VM). The skills transfer; the client's specifics never should.

1. Why Sophos changes the architecture, not just adds a component

Before: an endpoint-only lab (Sysmon/Wazuh) with MISP enrichment. There was no sensor watching the network boundary — everything had to already be running on the Windows/Linux VM to be seen at all.

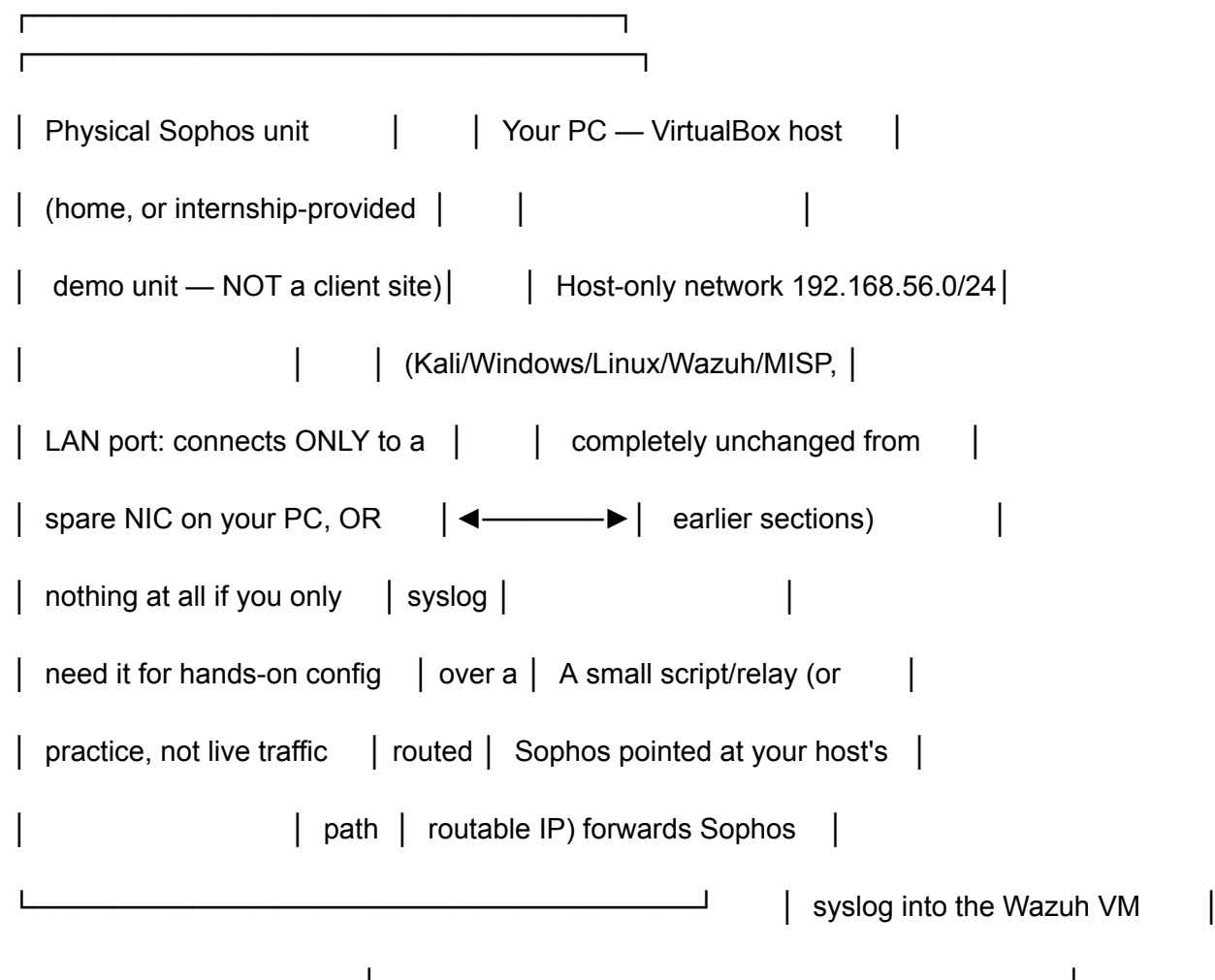
After: a genuine three-tier pipeline —

- **Perimeter** (Sophos) — sees things before they ever touch an endpoint: scan attempts, VPN brute force, IPS signature hits on traffic that never even reaches a host.
 - **Endpoint** (Sysmon/auditd + Wazuh agent) — unchanged from the earlier sections.
 - **Correlation + Intel** (Wazuh manager + MISP) — now correlates *two* independent telemetry sources, not one, which is what makes an alert more trustworthy (an IP that's both port-scanning at the perimeter *and* triggering an endpoint alert is a much stronger signal than either alone).
-

2. Hardware Integration — two real options, pick based on what you have

Do not skip this decision and just plug the Sophos LAN port into your normal home network switch. That would put your Kali attack VM's traffic — and anything Sophos itself does — on the same broadcast domain as your actual devices (phone, laptop, smart TV, etc). Pick one of these two instead.

Option A — Recommended default: keep the VM lab fully virtual, Sophos as an external log source only



You get: full hands-on Sophos config practice (rules, zones, IPS, VPN — the actual internship skill) **plus** real syslog data flowing into Wazuh, without ever bridging your isolated attack lab

onto a network segment the Sophos device's WAN side can reach. This is the option assumed by the rest of this section.

Option B — Advanced: a genuinely separate physical segment, for a closer-to-real topology

Only if you have a **managed, VLAN-capable switch** and a spare NIC on your host:

Internet — Sophos WAN

Sophos LAN — dedicated VLAN on managed switch — spare physical NIC (bridged adapter in VirtualBox)

— Kali/Windows/Linux VMs now sit on THIS segment

Wazuh/MISP VMs can stay on host-only (192.168.56.0/24) — Wazuh just needs L3 reachability to receive syslog, it doesn't need to be on the same segment as Sophos's LAN.

This is closer to a real deployment (Sophos genuinely routes/firewalls the attack traffic, not just receives its logs passively) but requires hardware most students don't have. If you don't have a managed switch you're confident configuring VLANs on, **use Option A** — a slightly less realistic topology you can verify is safe beats a more realistic one you're not sure is isolated correctly.

3. Sophos-side configuration

3.1 Initial setup (skip if already done as part of your internship training)

Basic zone/interface/rule concepts — these are the actual skills your offer letter names, and they're identical whether you're on a physical XGS unit or the free Home Edition VM:

1. **Configure** → **Network** → **Interfaces** — assign WAN/LAN roles to physical (or virtual) ports.
2. **Configure** → **Firewall Rules** — create at least one explicit LAN→WAN allow rule and observe the default "Deny" behavior for anything not explicitly matched (this default-deny is what detection 10.19 in [03-Detection-Engineering.md](#) keys off of).
3. **Protect** → **Intrusion Prevention** — enable a default IPS policy on the LAN→WAN rule (this is what generates the log data for a future IPS-based detection, listed as a natural extension in Section 21).

3.2 Configure syslog export to Wazuh

1. **System services** → **Log settings** → **Add** (under Syslog servers).
 2. Fill in:
 - **Name:** `wazuh-soc`
 - **IP address/Domain:** `192.168.56.5` (or your Wazuh VM's actual reachable IP under Option A/B above — adjust to your real topology, this address assumes the host-only scheme from [README.md §5.1](#))
 - **Port:** `5514` (kept distinct from any other syslog traffic on your network)
 - **Facility:** `DAEMON`
 - **Severity level:** `Information`
 - **Format:** **Device standard format** — not "Standard Syslog Format." This is the key-value format (`device="SFW" ... log_type="..."` ...) the decoder below is built against; the other format serializes fields differently (confirmed during this review — at least one vendor's documented example under "Standard Syslog Format" quotes `src_ip` differently than "Device standard format" does).
 3. Save, then under **Log type**, enable at minimum: **Firewall, IPS, Authentication/Admin & system events, VPN**. Add others once this narrow set is confirmed working.
-

4. Wazuh-side: listener, decoder, verification

4.1 Open the syslog listener

Edit `/var/ossec/etc/ossec.conf` on the manager, inside `<ossec_config>`:

```
<remote>

<connection>syslog</connection>

<port>5514</port>

<protocol>udp</protocol>

<allowed-ips>192.168.56.1/32</allowed-ips>

<local_ip>0.0.0.0</local_ip>

</remote>
```

Restart: `sudo systemctl restart wazuh-manager`

4.2 Capture a real raw log line before writing/trusting any decoder

```
sudo tcpdump -i eth0 src 192.168.56.1 and port 5514 -nn -A -c 5
```

Do not skip this. Everything below is built and cross-checked against real Sophos syslog samples gathered from Sophos's own community documentation and independent SIEM-vendor integration docs — but SFOS field formatting has genuinely shifted across firmware versions (18.x through 22.x). Your captured line is the ground truth, not this document. Check [System → About](#) on your unit for the exact SFOS version, and compare against docs.sophos.com/nsg/sophos-firewall/<your-version>/syslog/ if anything below doesn't match.

4.3 Decoder — corrected

What changed from the earlier draft, and why: the original decoder used five *same-named* sibling decoders, each trying to extract one field independently. Per Wazuh's own decoder documentation and a confirmed community troubleshooting thread with the identical structural problem, this doesn't reliably populate all five fields — Wazuh's guidance is to combine everything into one regex per log-format variant. Different Sophos log families (auth vs. firewall-rule vs. IPS) also have genuinely different field layouts (e.g., IPS/IDP logs omit the `status=` field that auth and firewall-rule logs both have), so one universal regex across all Sophos log types isn't realistic either — this decoder uses one parent + two purpose-built children instead, matching the proven pattern from real-world community Wazuh/pfSense decoders.

Create/edit `/var/ossec/etc/decoders/local_decoder.xml`:

```
<!-- Parent: matches any Sophos Firewall syslog line -->
```

```
<decoder name="sophos-firewall">
```

```
  <prematch>device="SFW"</prematch>
```

```
</decoder>
```

```
<!-- Child: Firewall/Admin authentication events (WebAdmin, User Portal, local CLI auth) -->
```

```
<decoder name="sophos-auth">
```

```
  <parent>sophos-firewall</parent>
```

```
<regex type="pcre2">log_type="([^\"]+)" log_component="(Firewall Authentication)"
log_subtype="([^\"]+)" status="([^\"]+)" priority=\S+
user_name="([^\"]*)" .+?src_ip=(\d+\.\d+\.\d+\.\d+)</regex>
```

```
<order>sophos.log_type,sophos.log_component,sophos.log_subtype,sophos.status,dstuser,srcip</order>
```

```
</decoder>
```

```
<!-- Child: Firewall Rule allow/deny events -->
```

```
<decoder name="sophos-fwrule">
```

```
<parent>sophos-firewall</parent>
```

```
<regex type="pcre2">log_type="([^\"]+)" log_component="(Firewall Rule)" log_subtype="([^\"]+)"
status="([^\"]+)" .+?src_ip=(\d+\.\d+\.\d+\.\d+) src_country_code=\S*
dst_ip=(\d+\.\d+\.\d+\.\d+) .+?srczone="([^\"]*)"</regex>
```

```
<order>sophos.log_type,sophos.log_component,sophos.log_subtype,sophos.status,srcip,dstip,
sophos.srczone</order>
```

```
</decoder>
```

```
<!-- Child: VPN connection events — field name is a best-effort pattern, not a confirmed exact
string.
```

See AUDIT-LOG.md Pass 2, finding #10: no authoritative sample of a Sophos VPN
log_component

value was found during this review. Rather than guess one exact string (which would silently

fail to match if wrong, the same class of bug already found once in this project), this regex

matches log_component containing "VPN" in any casing/wording (e.g. "SSL VPN", "VPN",
"IPSec VPN")

— narrow this to an exact match once you've confirmed the real value with wazuh-logtest
(§4.4). -->

```
<decoder name="sophos-vpn">
```

```
<parent>sophos-firewall</parent>
```

```
<regex type="pcre2">log_type="([^\"]+)" log_component="([^\"]*[Vv][Pp][Nn][^\"]*)"
log_subtype="([^\"]+)" status="([^\"]+)" .+?src_ip=(\d+\.\d+\.\d+\.\d+)</regex>
```

```
<order>sophos.log_type,sophos.log_component,sophos.log_subtype,sophos.status,srcip</order>
```

```
</decoder>
```

Notes on choices made here, stated so you can verify them yourself:

- `type="pcre2"` is declared explicitly on all three children because the regexes use `.+?/.+` bridging patterns between quoted fields — Wazuh's default (non-PCRE2) regex engine is a POSIX-like engine and shouldn't be assumed to support every PCRE construct; declaring PCRE2 removes that ambiguity.
- `src_ip` and `dst_ip` are matched **unquoted** (`\d+\.\d+\.\d+\.\d+`, no surrounding quotes) — verified against real "Device standard format" samples from three independent sources. If your captured line (§4.2) shows them quoted instead, that's a real, version-specific difference — adjust the regex, don't assume this document is right over your own evidence.
- `sophos-fwrule` also captures `srczone` (added in this pass) — needed for the WAN-noise suppression rule in [06-Tuning-IR-Documentation.md](#) §15.2, which referenced this field before the decoder actually produced it. That gap is now closed; the two files are consistent again.
- `sophos-vpn` is a **new, explicitly best-effort** child decoder — its `log_component` capture is a loose pattern (matches anything containing "VPN"), not a hardcoded exact string, specifically because the real value wasn't confirmed against a sample during this review. This is a deliberate design choice to avoid the same failure mode as the original draft's `"CLI"` mismatch (finding #2 in [AUDIT-LOG.md](#)): a decoder that's *narrower than reality* silently matches nothing, while one that's *broader than necessary* at least fires and gives you real data to narrow down from. Tighten this once verified.

4.4 Verify before writing any rule

```
sudo /var/ossec/bin/wazuh-logtest
```

Paste your real captured line (§4.2) at the prompt. Confirm `sophos.log_component` and `sophos.status` actually appear as decoded output fields — **this exact verification step is what the earlier draft's own bug (finding #1 in the audit log) would have failed if anyone had run it before writing rules.** Do not proceed to rules until this passes.

4.5 Rules

The three Sophos-specific detections (perimeter auth failure, deny-burst/port-scan, VPN brute force) are defined in full — including MITRE mapping, severity, false-positive notes, and investigation steps — as detections **10.18**, **10.19**, and **10.20** in `03-Detection-Engineering.md`, so they live alongside the other 17 detections rather than being duplicated here. Add them to `/var/ossec/etc/rules/local_rules.xml`, restart the manager, and re-run §4.4's `wazuh-logtest` against the rule file too (it tests decoders *and* rules together when given a full rule-matching log line).

5. End-to-end test scenarios

Test	Trigger	Expected result	Validates
Firewall auth failure	Fail a Sophos WebAdmin login 3× from Kali	Rule 110106 alert, level 8, <code>sophos.log_component: Firewall Authentication</code>	Finding #2 fix (correct log_component)
Perimeter deny burst	From Kali, generate 15+ denied connections to the Sophos LAN interface in under 60s (e.g. a fast <code>nmap</code> sweep against a port range with no matching allow rule)	Rule 110109 fires repeatedly (low severity), rule 110110 escalates once the threshold is hit	Detection 10.19
VPN login (if configured)	Connect via SSL VPN through Sophos	Rule 110103 should fire (base match, level 3); confirm via <code>wazuh-logtest</code> that <code>sophos.log_component</code> genuinely contains "VPN" for your firmware before trusting the 110111	AUDIT-LOG.md Pass 2 — structurally complete now, semantically unverified by design (see <code>sophos-vpn</code> decoder notes above)

Test	Trigger	Expected result	Validates
		brute-force escalation built on top of it	
Negative control	A clean, allowed connection through Sophos	No auth/deny-burst alert fires	Confirms rules aren't over-firing

Screenshot all of these — this is your strongest "it actually works, and here's proof I verified it rather than assumed it" evidence for both the report and the viva.

6. What to verify yourself before submission (carried forward, still true)

- Your exact SFOS version's syslog field names — cross-check docs.sophos.com/nsg/sophos-firewall/<version>/syslog/ against your own captured line, not this document, if anything doesn't match.
 - Whether Wazuh 5.0 has reached GA by the time you build this (documentation.wazuh.com/current/release-notes) — if so, `/var/ossec/` paths throughout this entire project need to become `/var/wazuh-manager/` and the Filebeat-dependent steps in Section 7 of `02-Installation-and-SIEM-Setup.md` no longer apply as written.
 - If your internship gives you access to a real client Sophos unit at some point during the 30-day placement: that device's logs, rules, and topology stay out of this repo entirely, per your offer letter and per §0 of this section.
-

This completes the perimeter layer. Return to [README.md](#) for the full project map, or [AUDIT-LOG.md](#) for the complete findings this integration pass was built around.